(defined in `$sortFlags`). When we iterate over `$data` we call `sort()` with the appropriate flag then print out the newly sorted elements. Here's what each flag means.

`SORT_REGULAR` is the default behavior of `sort`: PHP's standard comparison rules are used. In PHP 8 this means that the elements are compared as if they were operands in an expression using the `<=>` operator which we've already seen earlier in the chapter when we looked at comparison with callbacks. The numbers sort as you would expect (-57 as the lowest value), with the string "a" as the largest value. This is because `'1' <=> 'a'` evaluates to `-1` (the right-hand operand is larger).

`SORT_NUMERIC` forces numeric comparison. Here "a" is placed alongside the number zero because PHP will coerce strings to 0. `1 <=> (int) 'a'` evaluates to `1` (the left-hand operand is larger).

`SORT_STRING` forces string comparison in ASCII alphabetical order. `'-57'` is considered to have a larger value than `'-5.7'` because of the third character in each string: `'.' <=> '7'` evaluates to `-1` because it comes first (and has a lower byte value) in the ascii code page.

There a few more flags available:

- `SORT_LOCALE_STRING` – string-based comparison, based on current locale (controlled by the built-in `setlocale()` function).
- `SORT_NATURAL` – string-based, using a "natural" ordering, see "Natural Sorting" later in this section.
- `SORT_FLAG_CASE` -combine with `SORT_STRING` or `SORT_NATURAL` (e.g., `SORT_FLAG_CASE|SORT_NATURAL`) using bitwise OR to compare strings case-insensitively.

By Size

`sort` and its companion functions (`asort()`, `ksort()`, `rsort()` et al.) are not limited to primitive types. They will also accept arrays containing composite types. A moderately useful application for this functionality could be to arrange a multi-dimensional array by the size of the sub-arrays:

```
$a = array_pad([0], rand(0, 10), 0);
$b = array_pad([0], rand(0, 10), 0);
$c = array_pad([0], rand(0, 10), 0);
```